IEEE Standard for Floating Point Numbers

V Rajaraman

Floating point numbers are an important data type in computation which is used extensively. Yet, many users do not know the standard which is used in almost all computer hardware to store and process these. In this article, we explain the standards evolved by The Institute of Electrical and Electronic Engineers in 1985 and augmented in 2008 to represent floating point numbers and process them. This standard is now used by all computer manufacturers while designing floating point arithmetic units so that programs are portable among computers.



V Rajaraman is at the Indian Institute of Science, Bengaluru. Several generations of scientists and engineers in India have learnt computer science using his lucidly written textbooks on programming and computer fundamentals. His current research interests are parallel computing and history of computing.

Introduction

There are two types of arithmetic which are performed in computers: integer arithmetic and real arithmetic. Integer arithmetic is simple. A decimal number is converted to its binary equivalent and arithmetic is performed using binary arithmetic operations. The largest positive integer that may be stored in an 8-bit byte is +127, if 1 bit is used for sign. If 16 bits are used, the largest positive integer is +32767 and with 32 bits, it is +2147483647, quite large! Integers are used mostly for counting. Most scientific computations are however performed using real numbers, that is, numbers with a fractional part. In order to represent real numbers in computers, we have to ask two questions. The first is to decide how many bits are needed to encode real numbers and the second is to decide how to represent real numbers using these bits. Normally, in numerical computation in science and engineering, one would need at least 7 to 8 significant digits precision. Thus, the number of bits needed to encode 8 decimal digits is approximately 26, as $\log_2 10 = 3.32$ bits are needed on the average to encode a digit. In computers, numbers are stored as a sequence of 8-bit bytes. Thus 32 bits (4 bytes) which is bigger than 26 bits is a logical size to use for real numbers. Given 32 bits to encode real

Keywords
Floating point numbers, rounding, decimal floating point numbers, IEEE 754-2008 Standard.

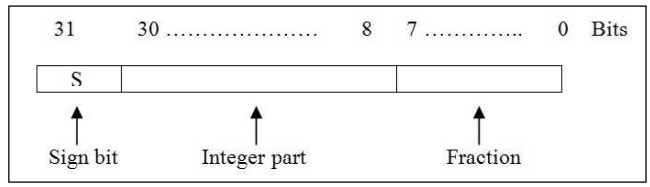


Figure 1. Fixed point representation of real numbers in binary using 32 bits.

numbers, the next question is how to break it up into an integer part and a fractional part. One method is to divide the 32 bits into 2 parts, one part to represent an integer part of the number and the other the fractional part as shown in *Figure* 1.

In this figure, we have arbitrarily fixed the (virtual) binary point between bits 7 and 8. With this representation, called *fixed point representation,* the largest and the smallest positive binary numbers that may be represented using 32 bits are

$$\text{Largest: } +\underbrace{111\ \ldots..}_{23\text{ bits}}\ \underbrace{1.11111111}_{8\text{ bits}} = 1677215.998046875$$

$$\text{Smallest: } +\underbrace{000\ \ldots}_{23\text{ bits}}\ \underbrace{0.00000001}_{8\text{ bits}} = 0.00390625$$

Binary Floating Point Numbers

This range of real numbers, when fixed point representation is used, is not sufficient in many practical problems. Therefore, another representation called *normalized floating point* representation is used for real numbers in computers. In this representation, 32 bits are divided into two parts: a part called the *mantissa* with its sign and the other called the *exponent* with its sign. The mantissa represents fractions with a non-zero leading bit and the exponent the power of 2 by which the mantissa is multiplied. This method increases the range of numbers that may be represented using 32 bits. In this method, a binary floating point number is represented by

As the range of real numbers representable with fixed point is not sufficient, normalized floating point is used to represent real numbers.

$$(\text{sign}) \times \text{mantissa} \times 2^{\pm \text{exponent}}$$

where the sign is one bit, the mantissa is a binary fraction with a non-zero leading bit, and the exponent is a binary integer. If 32

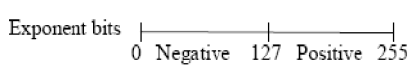
bits are available to store floating point numbers, we have to decide the following:

1. How many bits are to be used to represent the mantissa (1 bit is reserved for the sign of the number).

2. How many bits are to be used for the exponent.

3. How to represent the sign of the exponent.

The number of bits to be used for the mantissa is determined by the number of significant decimal digits required in computation, which is at least seven. Based on experience in numerical computation, it is found that at least seven significant decimal digits are needed if results are expected without too much error. The number of bits required to represent seven decimal digits is approximately 23. The remaining 8 bits are allocated for the exponent. The exponent may be represented in sign magnitude form. In this case, 1 bit is used for sign and 7 bits for the magnitude. The exponent will range from –127 to +127. The only disadvantage to this method is that there are two representations for 0 exponent: + 0 and – 0. To avoid this, one may use an *excess representation* or *biased format* to represent the exponent. The exponent has no sign in this format. The range 0 to 255 of an 8-bit number is divided into two parts 0 to 127 and 128 to 255 as shown below:



All bit strings from 0 to 126 are considered negative, exponent 127 represents 0, and values greater than 127 are positive. Thus the range of exponents is –127 to +128. Given an exponent string *exp*, the value of the exponent will be taken as (*exp* –127). The main advantage of this format is a unique representation for exponent zero. With the representation of binary floating point numbers explained above, the largest floating point number which can be represented is

The number of bits to be used for the mantissa is determined by the number of significant decimal digits required in computation, which is at least seven.

The main advantage of biased exponent format is unique representation for exponent 0.

$$0.1111\ldots1111 \times 2^{11111111}$$
$$|\leftarrow \text{ 23 bits } \rightarrow|$$
$$= (1 - 2^{-23}) \times 2^{255-127}$$
$$\cong 3.4 \times 10^{38}.$$

The smallest floating point number is

$$0.10000\ldots00 \times 2^{-127}$$
$$|\leftarrow \text{ 23 bits} \rightarrow|$$
$$\cong 0.293 \times 10^{-38} .$$

Example. Represent 52.21875 in 32-bit binary floating point format.

$52.21875 = 110100.00111 = .11010000111 \times 2^6$.
Normalized 23 bit mantissa = 0.11010000111000000000000.
As excess representation is being used for exponent, it is equal to $127 + 6 = 133$.
Thus the representation is
$52.21875 = 0.11010000111 \times 2^{133} = 0.110100000111 \times 2^{10000101}$.
The 32-bit string used to store 52.21875 in a computer will thus be 01000010111010000111000000000000.
Here, the most significant bit is the sign, the next 8 bits the exponent, and the last 23 bits the mantissa.

Institute of Electrical and Electronics Engineers standardized how floating point binary numbers would be represented in computers, how these would be rounded, how 0 and ∞ would be represented and how to treat exception condition such as attempt to divide by 0.

IEEE Floating Point Standard 754-1985

Floating point binary numbers were beginning to be used in the mid 50s. There was no uniformity in the formats used to represent floating point numbers and programs were not portable from one manufacturer's computer to another. By the mid 1980s, with the advent of personal computers, the number of bits used to store floating point numbers was standardized as 32 bits. A Standards Committee was formed by the Institute of Electrical and Electronics Engineers to standardize how floating point binary numbers would be represented in computers. In addition, the standard specified uniformity in rounding numbers, treating exception conditions such as attempt to divide by 0, and representation of 0 and infinity (∞). This standard, called IEEE Standard 754 for

floating point numbers, was adopted in 1985 by all computer manufacturers. It allowed porting of programs from one computer to another without the answers being different. This standard defined floating point formats for 32-bit and 64-bit numbers. With improvement in computer technology it became feasible to use a larger number of bits for floating point numbers. After the standard was used for a number of years, many improvements were suggested. The standard was updated in 2008. The current standard is IEEE 754-2008 version. This version retained all the features of the 1985 standard and introduced new standards for 16 and128-bit numbers. It also introduced standards for representing decimal floating point numbers. We will describe these standards later in this article. In this section, we will describe IEEE 754-1985 Standard.

The current standard is IEEE 754-2008 version. This version retained all the features of the 1985 standard and introduced new standards for 16-bit and128-bit numbers. It also introduced standards for representing decimal floating point numbers.

IEEE floating point representation for binary real numbers consists of three parts. For a 32-bit (called single precision) number, they are:

1. Sign, for which 1 bit is allocated.
2. Mantissa (called *significand* in the standard) is allocated 23 bits.
3. Exponent is allocated 8 bits. As both positive and negative numbers are required for the exponent, instead of using a separate sign bit for the exponent, the standard uses a biased representation. The value of the bias is 127. Thus an exponent 0 means that –127 is stored in the exponent field. A stored value 198 means that the exponent value is (198 – 127) = 71. The exponents –127 (all 0s) and + 128 (all 1s) are reserved for representing special numbers which we discuss later.

To increase the precision of the significand, the IEEE 754 Standard uses a normalized significand which implies that its most significant bit is always 1. As this is implied, it is assumed to be on the left of the (virtual) decimal point of the significand. Thus in the IEEE Standard, the significand is 24 bits long – 23 bits of the significand which is stored in the memory and an implied 1 as the most significant 24th bit. The extra bit increases the number

b_0	$b_1\ b_2\ b_3$ ..... b_8	$b_9\ b_{10}\ b_{11}$.....$b_{30}\ b_{31}$
Sign	Exponent	Significand

Figure 2. IEEE 754 representation of 32-bit floating point number.

of significant digits in the significand. Thus, a floating point number in the IEEE Standard is

$$(-1)^{s} \times (1.f)_2 \times 2^{\text{exponent} - 127}$$

where s is the sign bit; $s = 0$ is used for positive numbers and $s = 1$ for representing negative numbers. f represents the bits in the significand. Observe that the implied 1 as the most significant bit of the significand is explicitly shown for clarity.

For a 32-bit word machine, the allocation of bits for a floating point number are given in *Figure* 2. Observe that the exponent is placed before the significand (see *Box* 1).

Example. Represent 52.21875 in IEEE 754 – 32-bit floating point format.

$52.21875 = 110100.00111$

$= 1.1010000111 \times 2^5$.

Normalized significand = .1010000111.

Exponent: $(e - 127) = 5$ or, $e = 132$.

The bit representation in IEEE format is

0	10000100	10100001110000000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

Box 1.

Why does IEEE 754 standard use biased exponent representation and place it before the significand?

Computers normally have separate arithmetic units to compute with integer operands and floating point operands. They are called FPU (Floating Point Unit) and IU (Integer Unit). An IU works faster and is cheaper to build. In most recent computers, there are several IUs and a smaller number of FPUs. It is preferable to use IUs whenever possible. By placing the sign bit and biased exponent bits as shown in *Figure* 2, operations such as $x <r.o.> y$, where $r.o.$ is a relational operator, namely, $>, <, \geq, \leq$, and x, y are reals, can be carried out by integer arithmetic units as the exponents are integers. Sorting real numbers may also be carried out by IUs as biased exponent are the most significant bits. This ensures lexicographical ordering of real numbers; the significand need not be considered as a fraction.

Special Values in IEEE 754-1985 Standard (32 Bits)

***Representation of Zero*:** As the significand is assumed to have a hidden 1 as the most significant bit, all 0s in the significand part of the number will be taken as 1.00…0. Thus zero is represented in the IEEE Standard by all 0s for the exponent and all 0s for the significand. All 0s for the exponent is not allowed to be used for any other number. If the sign bit is 0 and all the other bits 0, the number is +0. If the sign bit is 1 and all the other bits 0, it is –0. Even though +0 and –0 have distinct representations they are assumed equal. Thus the representations of +0 and –0 are:

As the significand is assumed to have a hidden 1 as the most significant bit, all 0s in the significand part of the number will be taken as 1.00…0. Thus zero is represented in the IEEE Standard by all 0s for the exponent and all 0s for the significand.

+0

0	00000000	00000000000000000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

–0

1	00000000	00000000000000000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

***Representation of Infinity*:** All 1s in the exponent field is assumed to represent infinity (∞). A sign bit 0 represents +∞ and a sign bit 1 represents –∞. Thus the representations of +∞ and –∞ are:

+∞

0	11111111	00000000000000000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

–∞

1	11111111	00000000000000000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

All 1s in the exponent field is assumed to represent infinity (∞).

***Representation of Non Numbers*:** When an operation is performed by a computer on a pair of operands, the result may not be mathematically defined. For example, if zero is divided by zero,

When an arithmetic operation is performed on two numbers which results in an indeterminate answer, it is called NaN (Not a Number) in IEEE Standard.

the result is indeterminate. Such a result is called Not a Number (NaN) in the IEEE Standard. In fact the IEEE Standard defines two types of NaN. When the result of an operation is not defined (i.e., indeterminate) it is called a Quiet NaN (QNaN). Examples are: 0/0, $(\infty - \infty)$, $\sqrt{-1}$. Quiet NaNs are normally carried over in the computation. The other type of NaN is called a Signalling Nan (SNaN). This is used to give an error message. When an operation leads to a floating point underflow, i.e., the result of a computation is smaller than the smallest number that can be stored as a floating point number, or the result is an overflow, i.e., it is larger than the largest number that can be stored, SNaN is used. When no valid value is stored in a variable name (i.e., it is undefined) and an attempt is made to use it in an arithmetic operation, SNaN would result. QNaN is represented by 0 or 1 as the sign bit, all 1s as exponent, and a 0 as the left-most bit of the significand and at least one 1 in the rest of the significand. SNaN is represented by 0 or 1 as the sign bit, all 1s as exponent, and a 1 as the left-most bit of the significand and any string of bits for the remaining 22 bits. We give below the representations of QNaN and SNaN.

QNaN

0 or 1	11111111	00010000000000000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

The most significant bit of the significand is 0. There is at least one 1 in the rest of the significand.

SNaN

0 or 1	11111111	10000000000001000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

The most significant bit of the significand is 1. Any combination of bits is allowed for the other bits of the significand.

***Largest and Smallest Positive Floating Point Numbers*:**

Largest Positive Number

0	11111110	11111111111111111111111
Sign 1 bit	Exponent 8 bits	Significand 23 bits

Significand: $1111 \ldots 1 = 1 + (1 - 2^{-23}) = 2 - 2^{-23}$.
Exponent: $(254 - 127) = 127$.
Largest Number $= (2 - 2^{-23}) \times 2^{127} \cong 3.403 \times 10^{38}$.
If the result of a computation exceeds the largest number that can be stored in the computer, then it is called an *overflow*.

Smallest Positive Number

0	00000001	00000000000000000000000
Sign 1 bit	Exponent 8 bits	Significand 23 bits

Significand = 1.0.
Exponent = 1 – 127 = –126.
The smallest normalized number is $= 2^{-126} \cong 1.1755 \times 10^{-38}$.

***Subnormal Numbers*:** When all the exponent bits are 0 and the leading hidden bit of the siginificand is 0, then the floating point number is called a *subnormal number*. Thus, one logical representation of a subnormal number is

$$(-1)^s \times 0.f \times 2^{-127} \text{ (all 0s for the exponent)},$$

where f has at least one 1 (otherwise the number will be taken as 0). However, the standard uses –126, i.e., bias +1 for the exponent rather than –127 which is the bias for some not so obvious reason, possibly because by using –126 instead of –127, the gap between the largest subnormal number and the smallest normalized number is smaller.

The largest subnormal number is $0.999999988 \times 2^{-126}$. It is close to the smallest normalized number 2^{-126}.

When all the exponent bits are 0 and the leading hidden bit of the siginificand is 0, then the floating point number is called a *subnormal number.*

By using subnormal numbers, underflow which may occur in some calculations are gradual.

The smallest positive subnormal number is

0	00000000	00000000000000000000001
Sign 1 bit	Exponent 8 bits	Significand 23 bits

the value of which is $2^{-23} \times 2^{-126} = 2^{-149}$.

A result that is smaller than the smallest number that can be stored in a computer is called an *underflow*.

You may wonder why subnormal numbers are allowed in the IEEE Standard. By using subnormal numbers, underflow which may occur in some calculations are gradual. Also, the smallest number that could be represented in a machine is closer to zero. (See *Box* 2.)

Box 2. Machine Epsilon and Dwarf

In any of the formats for representing floating point numbers, *machine epsilon* is defined as the difference between 1 and the next larger number that can be stored in that format. For example, in 32-bit IEEE format with a 23-bit significand, the machine epsilon is $2^{-23} = 1.19 \times 10^{-7}$. This essentially tells us that the precision of decimal numbers stored in this format is 7 digits. The term precision and accuracy are not the same. Accuracy implies correctness whereas precision does not. For 64-bit representation of IEEE floating point numbers the significand length is 52 bits. Thus, machine epsilon is $2^{-52} = 2.22 \times 10^{-16}$. Therefore, decimal calculations with 64 bits give 16 digit precision. This is a conservative definition used by industry. In MATLAB, machine epsilon is as defined above. However, academics define machine epsilon as the upper bound of the relative error when numbers are rounded. Thus, for 32-bit floating point numbers, the machine epsilon is $2^{-23}/2 \cong 5.96 \times 10^{-8}$.

The machine epsilon is useful in iterative computation. When two successive iterates differ by less than |epsilon| one may assume that the iteration has converged. The IEEE Standard does not define machine epsilon.

Tiny or dwarf is the minimum subnormal number that can be represented using the specified floating point format. Thus, for IEEE 32-bit format, it is

0	00000000	00000000000000000000001
Sign 1 bit	Exponent 8 bits	Significand 23 bits

whose value is: $2^{-126-23} = 2^{-149} \cong 1.4 \times 10^{-45}$. Any number less than this will signal underflow. The advantage of using subnormal numbers is that underflow is gradual as was stated earlier.

When a mathematical operation such as add, subtract, multiply, or divide is performed with two floating point numbers, the significand of the result may exceed 23 bits after the adjustment of the exponent. In such a case, there are two alternatives. One is to truncate the result, i.e., ignore all bits beyond the 32nd bit. The other is to round the result to the nearest significand. For example, if a significand is 0.110 .. 01 and the overflow bit is 1, a rounded value would be 0.111 … 10. In other words, if the first bit which is truncated is 1, add 1 to the least significant bit, else ignore the bits. This is called *rounding upwards*. If the extra bits are ignored, it is called *rounding downwards*. The IEEE Standard suggests to hardware designers to get the best possible result while performing arithmetic that are *reproducible* across different manufacturer's computers using the standard. The Standard suggests that in the equation

$$c = a \ <op> \ b,$$

where a and b are operands and $<op>$ an arithmetic operation, the result c should be as if it was computed exactly and then rounded. This is called *correct rounding*.

In *Tables* 1 and 2, we summarise the discussions in this section.

Table 1. IEEE 754-85 floating point standard. We use f to represent the significand, e to represent the exponent, b the bias of the exponent, and ± for the sign.

Value	Sign	Exponent (8 bits)	Significand (23 bits)
+0	0	00000000	00 ….00 (all 23 bits 0)
− 0	1	00000000	00 ….00 (all 23 bits 0)
$+ 1.f \times 2^{(e-b)}$ e exponent, b bias	0	00000001to 11111110	$a\ a$ …. $a\ a$ (a = 0 or 1)
$- 1.f \times 2^{(e-b)}$	1	00000001to 11111110	$a\ a$ …. $a\ a$ (a = 0 or 1)
+ ∞	0	11111111	000 … 00 (all 23 bits 0)
− ∞	1	11111111	000 … 00 (all 23 bits 0)
SNaN	0 or 1	11111111	000 … 01 to 011… 11 leading bit 0 (at least one 1 in the rest)
QNaN	0 or 1	11111111	1000 … 10 leading bit 1
Positive subnormal $0.f \times 2^{x+1-b}$ (x is the number of leading 0s in significand)	0	00000000	000 … 01 to 111… 11 (at least one 1)

Table 2. Operations on special numbers. All NaNs are Quiet NaNs.

Operation	Result	Operation	Result
$n / \pm \infty$	0	$\pm 0 / \pm 0$	NaN
$\pm \infty / \pm \infty$	$\pm \infty$	$\infty - \infty$	NaN
$\pm n / 0$	$\pm \infty$	$\pm \infty / \pm \infty$	NaN
$\infty + \infty$	∞	$\pm \infty \times 0$	NaN

IEEE 754 Floating Point 64-Bit Standard – 1985

The IEEE 754 floating point standard for 64-bit (called double precision) numbers is very similar to the 32-bit standard. The main difference is the allocation bits for the exponent and the significand. With 64 bits available to store floating point numbers, there is more freedom to increase the significant digits in the significand and increase the range by allocating more bits to the exponent. The standard allocates 1 bit for sign, 11 bits for the exponent, and 52 bits for the significand. The exponent uses a biased representation with a bias of 1023. The representation is shown below:

With 64 bits available to store floating point numbers, there is more freedom to increase the significant digits in the significand and increase the range by allocating more bits to the exponent. The standard allocates 1 bit for sign, 11 bits for the exponent, and 52 bits for the significand.

Sign	Exponent	Significand
1 bit	11 bits	52 bits

A number in this standard is thus

$$(-1)^s \times (1.f) \times 2^{(\text{exponent} - 1023)},$$

where $s = \pm 1$, and f is the value of the significand.

The largest positive number which can be represented in this standard is

0	11111111110	1111.....1111
Sign 1 bit	Exponent 11 bits	Significand 52 bits

which is $= (2 - 2^{-52}) \times 2^{(2046 - 1023)} = (2 - 2^{-52}) \times 2^{1023} \cong 10^{3083}$. The smallest positive number that can be represented using 64

bits using this standard is

0	00000000001	0000..... 00000
Sign 1 bit	Exponent 11 bits	Significand 52 bits

whose value is $2^{1-1023} = 2^{-1022}$.

The definitions of ± 0, $\pm \infty$, QNaN, SNaN remain the same except that the number of bits in the exponent and significand are now 11 and 52 respectively. Subnormal numbers have a similar definition as in 32-bit standard.

IEEE 2008 Standard has introduced standards for 16-bit and 128-bit floating point numbers. It has also introduced standards for floating point decimal numbers.

IEEE 754 Floating Point Standard – 2008

This standard was introduced to enhance the scope of IEEE 754 floating point standard of 1985. The enhancements were introduced to meet the demands of three sets of professionals:

1. Those working in the graphics area.
2. Those who use high performance computers for numeric intensive computations.
3. Professionals carrying out financial transactions who require exact decimal computation.

This standard has not changed the format of 32- and 64-bit numbers. It has introduced floating point numbers which are 16 and 128 bits long. These are respectively called half and quadruple precision. Besides these, it has also introduced standards for floating point decimal numbers.

The 16-Bit Standard

The 16-bit format for real numbers was introduced for pixel storage in graphics and not for computation. (Some graphics processing units use the 16-bit format for computation.) The 16-bit format is shown in *Figure* 3.

The 16-bit format for real numbers was introduced for pixel storage in graphics and not for computation.

b_0	$b_1\ b_2\ b_3\ b_4\ b_5$	$b_6\ b_7\ b_8\ b_9\ b_{10}$ .... $b_{14}\ b_{15}$
Sign	Exponent	Significand
1 bit	5 bits	10 bits

Figure 3. Representation of 16-bit floating point numbers.

With 16-bit representation, 5 bits are used for the exponent and 10 bits for the significand. Biased exponent is used with a bias of 15. Thus the exponent range is: –14 to +15. Remember that all 0s and all 1s for the exponent are reserved to represent 0 and ∞, respectively.

The maximum positive integer that can be stored is

$$+ 1.111..1 \times 2^{15} = 1 + (1 - 2^{-11}) \times 2^{15} = (2 - 2^{-11}) \times 2^{15} \cong 65504.$$

The minimum positive number is

$$+ 1.000 \ldots 0 \times 2^{-14} = 2^{-14} = 0.61 \times 10^{-4}.$$

The minimum subnormal 16-bit floating point number is

$$2^{-24} \cong 5.96 \times 10^{-8}.$$

The definitions of ± 0, ± ∞, NaN, and SNaN follow the same ideas as in 32-bit format.

With improvements in computer technology, using 128 bits to represent floating point numbers has become feasible. This is sometimes used in numeric-intensive computation, where large rounding errors may occur.

The 128-Bit Standard

With improvements in computer technology, using 128 bits to represent floating point numbers has become feasible. This is sometimes used in numeric-intensive computation, where large rounding errors may occur. In this standard, besides a sign bit, 14 bits are used for the exponent, and 113 bits are used for the significand. The representation is shown in *Figure* 4.

A number in this standard is

$$(-1)^s \times (1.f) \times 2^{(exponent - 16384)}.$$

Sign	Exponent	Significand
1 bit	14 bits	113 bits

Figure 4. Representation of 128-bit floating point number.

The largest positive number which can be represented in this format is:

0	11111111111110	111……111
Sign 1 bit	Exponent 14 bits	Significand 113 bits

which equals $(1 - 2^{-113}) \times 2^{16383} = \cong 10^{4932}$.

The smallest normalized positive number which can be represented is

$$2^{1-16383} = 2^{-16382} \cong 10^{-4931}.$$

A subnormal number is represented by

$$(-1)^s \times 0.f \times 2^{-16382}.$$

The smallest positive subnormal number is

$$2^{-16382-113} = 2^{-16485}.$$

The definitions of ± 0, $\pm \infty$, and QNaN, and SNaN are the same as in the 1985 standard except for the increase of bits in the exponent and significand. There are other minor changes which we will not discuss.

The Decimal Standard

The main motivation for introducing a decimal standard is the fact that a terminating decimal fraction need not give a terminating binary fraction. For example, $(0.1)_{10}$ = (0.00011 (0011) recurring)$_2$. If one computes 100000000 × 0.1 using binary arithmetic and rounding the non-terminating binary fraction up to the next larger number, the answer is: 10000001.490116 instead of 10000000. This is unacceptable in many situations, particularly in financial transactions. There was a demand from professionals carrying out financial transactions to introduce a standard for representing decimal floating point numbers. Decimal floating point numbers are not new. They were available in COBOL. There was, however, no standard and this resulted in non-portable programs. IEEE 754-2008 has thus introduced a standard for

The main motivation for introducing a decimal standard is the fact that a terminating decimal fraction need not give a terminating binary fraction.

There was a demand from professionals carrying out financial transactions to introduce a standard for representing decimal floating point numbers.

decimal floating point numbers to facilitate portability of programs.

Decimal Representation

The idea of encoding decimal numbers using bits is simple. For example, to represent 0.1 encoded (not converted) to binary, it is written as 0.1×10^0. The exponent instead of being interpreted as a power of 2 is now interpreted as a power of 10. Both the significand and the exponent are now integers and their binary representations have a finite number of bits. One method of representing 0.1×10^0 using this idea in a 32-bit word is shown in *Figure* 5.

We have assumed an 8-bit binary exponent in biased form. In this representation, there is no hidden 1 in the significand. The significand is an integer. (Fractions are represented by using an appropriate power of 10 in the exponent field.) For example, .000753658 will be written as 0.753658×10^{-3} and 753658, an integer, will be converted to binary and stored as the significand. The power of 10, namely –3, will be an integer in the exponent part.

The IEEE Standard does not use this simple representation. This is due to the fact that the largest decimal significand that may be represented using this method is 0.8388607.

It is preferable to obtain 7 significant digits, i.e., significand up to 0.9999999. The IEEE Standard achieves this by borrowing some bits from the exponent field and reducing the range of the exponent. In other words, instead of maximum exponent of 128, it is reduced to 96 and the bits saved are used to increase the significant digits in the significand. The coding of bits is as shown in *Figure* 6.

Figure 5. Representation of decimal floating point numbers.

0	01111110	00000000000000000000001
Sign	Exponcnt	Significand
1 bit	8 bits	23 bits

Figure 6. Representation of 32-bit floating point numbers in IEEE 2008 Standard.

Sign	Combination Bits	Exponent Continuation	Coefficient
1 bit	5 bits	6 bits	20 bits

Observe the difference in terminology for exponent and significand fields. In this coding, one part of the five combination bits is interpreted as exponent and the other part as an extension of significand bits using an ingenious method. This will be explained later in this section.

IEEE 2008 Standard defines two formats for representing decimal floating point numbers. One of them *converts* the binary significand field to a decimal integer between 0 and 10^{p-1} where p is the number of significant bits in the significand. The other, called densely packed decimal form, *encodes* the bits in the significand field directly to decimal. Both methods represent decimal floating point numbers as shown below.

Decimal floating point number = $(-1)^s \times f \times 10^{exp-bias}$, where s is the sign bit, f is a decimal fraction (significand), *exp* and *bias* are decimal numbers. The fraction need not be normalized but is usually normalized with a non-zero most significant digit. There is no hidden bit as in the binary standard. The standard defines decimal floating point formats for 32-bit, 64-bit and 128-bit numbers.

IEEE 2008 Standard defines two formats for representing decimal floating point numbers. One of them *converts* the binary significand field to a decimal integer between 0 and 10^{p-1} where p is the number of significant bits in the significand. The other, called densely packed decimal form, *encodes* the bits in the significand field directly to decimal.

Densely Packed Decimal Significand Representation

Normally, when decimal numbers are coded to binary, 4 bits are required to represent each digit, as 3 bits give only 8 combinations that are not sufficient to represent 10 digits. Four bits give 16 combinations out of which we need only 10 combinations to represent decimal numbers from 0 to 9. Thus, we waste 6 combinations out of 16. For a 32-bit number, if 1 bit is used for sign, 8 bits for the exponent and 23 bits for the significand, the maximum positive value of the significand will be +0.799999. The exponent range will be 00 to 99. With a bias of 50, the maximum positive decimal number will be 0.799999×10^{49}.

The number of significand digits in the above coding is not sufficient. A clever coding scheme for decimal encoding of binary numbers was discovered by Cowlinshaw. This method encodes 10 bits as 3 decimal digits (remember that $2^{10} = 1024$ and

one can encode 000 to 999 using the available 1024 combinations of 10 bits). IEEE 754 Standard uses this coding scheme. We will now describe the standard using this coding method for 32-bit numbers.

The standard defines the following:

1. A *sign bit* 0 for + and 1 for – .
2. A *combination field* of 5 bits. It is called a combination field as one part of it is used for the exponent and another part for the significand.
3. An *exponent continuation field* that is used for the exponent.
4. A *coefficient field* that encodes strings of 10 bits as 3 digits. Thus, 20 bits are encoded as 6 digits. This is part of the significand field.

For 32-bit IEEE word, the distribution of bits was given in *Figure* 6 which is reproduced below for ready reference.

Sign	Combination field	Exponent continuation field	Coefficient field
1 bit	5 bits	6 bits	20 bits

The 5 bits of the combination field are used to increase the coefficient field by 1 digit. It uses the first ten 4-bit combinations of 16 possible 4-bit combinations to represent this. As there are 32 combinations of 5 bits, out of which only 10 are needed to extend the coefficient digits, the remaining 22-bit combinations are available to increase the exponent range and also encode ∞ and NaN. This is done by an ingenious method explained in *Table* 3.

Table 3. Use of combination bits in IEEE Decimal Standard.

Combination bits $b_1\ b_2\ b_3\ b_4\ b_5$	**Type**	**Most significant bits of exponent**	**Most significant digit of coefficient**
x y a b c	Finite	x y	0 a b c
1 1 a b c	Finite	a b	1 0 0 c
1 1 1 1 0	∞	–	–
1 1 1 1 1	NaN	–	–

Sign bit	1	1
Combination field (bits)	5	5
Exponent continuation field (bits)	8	12
Coefficient field (bits)	50	110
Number of bits in number	64	128
Significant digits	16	34
Exponent range (decimal)	768	12288
Exponent bias (decimal)	384	6144

Table 4. Decimal representation for 64-bit and 128-bit numbers.

Thus, the most significant 2 bits of the exponent are constrained to take on values 00, 01, 10. Using the 6 bits of exponent combination bits, we get an exponent range of $3 \times 2^6 = 192$. The exponent bias is 96. (Remember that the exponent is an integer that can be exactly represented in binary.) With the addition of 1 significant digit to the 6 digits of the coefficient bits, the decimal representation of 32-bit numbers has 7 significant digits. The largest number that can be stored is 0.9999999×10^{96}. A similar method is used to represent 64- and 128-bit floating point numbers. This is shown in *Table* 4.

In the IEEE 2008 Decimal Standard, the combination bits along with exponent bits are used to represent NaN, signaling NaN and $\pm \infty$ as shown in *Table* 5.

Quantity	**Sign bit**	**Combination bits**
	b_0	$b_1\ b_2\ b_3\ b_4\ b_5\ b_6$
SNaN	0 or 1	1 1 1 1 1 1
QNaN	0 or 1	1 1 1 1 1 0
$+\infty$	0	1 1 1 1 0 x
$-\infty$	1	1 1 1 1 0 x
+ 0	0	0 0 0 0 0 x
– 0	1	0 0 0 0 0 x
		(x is 0 or 1)

Table 5. Representation of NaNs, $\pm \infty$, and $\pm$ 0.

Binary Integer Representation of Significand

In this method, as we stated earlier, if there are p bits in the extended coefficient field constituting the significand, they are converted as a decimal integer. This representation also uses the bits of the combination field to increase the number of significant digits. The idea is similar and we will not repeat it. The representation of NaNs and $\pm\infty$ is the same as in the densely packed format. The major advantage of this representation is the simplicity of performing arithmetic using hardware arithmetic units designed to perform arithmetic using binary numbers. The disadvantage is the time taken for conversion from integer to decimal. The densely packed decimal format allows simple encoding of binary to decimal but needs a specially designed decimal arithmetic unit.

Acknowledgements

I thank Prof. P C P Bhatt, Prof. C R Muthukrishnan, and Prof. S K Nandy for reading the first draft of this article and suggesting improvements.

Suggested Reading

[1] David Goldberg, What every computer scientist should know about floating point arithmetic, *ACM Computing Surveys,* Vol.28, No.1, pp5–48, March 1991.

[2] V Rajaraman, *Computer Oriented Numerical Methods*, 3rd Edition, PHI Learning, New Delhi, pp.15–32, 2013.

[3] Marc Cowlinshaw, Densely packed decimal encoding, *IEEE Proceedings – Computer and Digital Techniques,* Vol.149, No.3, pp.102-104, May 2002.

[4] Steve Hollasch, IEEE Standard 754 for floating point numbers, steve.hollasch.net/cgindex/coding/ieeefloat.html

[5] Decimal Floating Point, Wikipedia, en.wikipedia.org/wiki/Decimal_floating_point

Address for Correspondence
V Rajaraman
Supercomputer Education and Research Centre
Indian Institute of Science
Bengaluru 560 012, India.
Email: rajaram@serc.iisc.in